

# CPE 486/586: Deep Learning for Engineering Applications

10 Physics-informed Neural Networks

Spring 2026

**Rahul Bhadani**

# Outline

1. Motivation
2. Goal
3. Solving Nonlinear Partial Differential Equations

# Motivation

# Solving a Differential Equation

Consider a differential equation:

$$\frac{dx(t)}{dt} = -2x$$

How do we solve this Equation?

# Solving a Differential Equation

Consider a differential equation:

$$\frac{dx(t)}{dt} = -2x$$

How do we solve this Equation?

Analytically, it is  $x = e^{-2t}$  with initial condition  $x(0) = 1$ .

# Solving a Differential Equation

Consider a differential equation:

$$\frac{dx(t)}{dt} = -2x$$

How do we solve this Equation?

Analytically, it is  $x = e^{-2t}$  with initial condition  $x(0) = 1$ .

But it is usually not analytically possible to solve all kind of differential equations.

## Now Data Enters the Picture

Now, think of a situation when we know  $x$  from some measurement of the process and also assume that it follows the differential equation provided earlier.

## Now Data Enters the Picture

Now, think of a situation when we know  $x$  from some measurement of the process and also assume that it follows the differential equation provided earlier.

Here, the independent variable is  $t$ , we know  $x$  and we want to find  $f$  such that it can give  $x$  for any value of  $t$ .



## Now Data Enters the Picture

Now, think of a situation when we know  $x$  from some measurement of the process and also assume that it follows the differential equation provided earlier.

Here, the independent variable is  $t$ , we know  $x$  and we want to find  $f$  such that it can give  $x$  for any value of  $t$ .

What's the problem then?

## Now Data Enters the Picture

Now, think of a situation when we know  $x$  from some measurement of the process and also assume that it follows the differential equation provided earlier.

Here, the independent variable is  $t$ , we know  $x$  and we want to find  $f$  such that it can give  $x$  for any value of  $t$ .

What's the problem then?

It may not necessarily be the solution to the differential equation provided earlier!

# Differential Equation as Regularizer

Let's rewrite our differential equation

Consider a differential equation:

$$\frac{dx(t)}{dt} + 2x = 0$$

But as the real world data will exhibit some error,  $x(t)$  might not satisfy the above equation. But if we approximate  $x(t)$  using some neural network  $u^\theta(x, t)$ , we would want

$$\frac{du^\theta(x, t)}{dt} + 2u^\theta(x, t) \leq \varepsilon$$

where  $\varepsilon$

# Differential Equation as Regularizer

Hence, apart from the training loss, we introduce a physics loss

Consider a differential equation:

$$\mathcal{L}_{physics}$$

which is

$$\mathcal{L}_{physics} = \frac{du^{\theta}(x, t)}{dt} + 2u^{\theta}(x, t)$$

for the previous example that we would be looking to minimize.

This is what we call **Physics-Informed Deep Learning (PIDL)** or in general **Physics Informed Machine Learning (PIML)** and the Neural Network used would be called **Physics-Informed Neural Network (PINN)**.

# Updated Loss Function

$$\mathcal{L} = \lambda_1 \mathcal{L}_{data} + \lambda_2 \mathcal{L}_{physics} + \lambda_3 \mathcal{L}_{IC} + \lambda_4 \mathcal{L}_{BC}$$

where  $IC$  is for initial conditions and  $BC$  is for boundary conditions.

Physics governs almost every system we want to model.

**Classical numerical solvers** (FEM, FDM, FVM):

- ⚡ Require fine meshes
- ⚡ Scale poorly to high dimensions
- ⚡ Cannot easily solve inverse problems
- ⚡ Need full boundary conditions

**Pure data-driven ML:**

- ⚡ Needs enormous labelled data
- ⚡ No physics guarantee
- ⚡ Fails to extrapolate
- ⚡ Black-box predictions

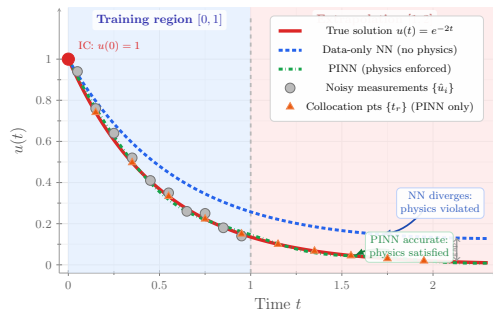
**Can we get the best of both worlds?**

# Data Alone is Not Enough

Suppose we fit a neural network  $u^\theta(x, t)$  purely on noisy measurements  $\{(t_i, \hat{x}_i)\}$ .

## Problems:

- ⚡ The network may **overfit** to noise
- ⚡ **Extrapolation** outside training region fails
- ⚡ The prediction is **not guaranteed** to satisfy  $\frac{dx}{dt} = -2x$
- ⚡ Different networks with the same data error can give **physically inconsistent** results



Red: truth  
PINN

Blue: data-only NN

Green:

# The Complete PINN Loss

We want  $u^\theta(x, t)$  to simultaneously:

- 1 **Fit the data** at measurement points  $\{t_i, \hat{x}_i\}$
- 2 **Satisfy the ODE** at collocation points  $\{t_r\}$
- 3 **Satisfy the initial condition**  $u^\theta(x, 0) = x_0$

$$\mathcal{L}_{\text{total}}(\theta) = \underbrace{\lambda_d \frac{1}{N_d} \sum_{i=1}^{N_d} |u^\theta(x, t_i) - \hat{x}_i|^2}_{\mathcal{L}_{\text{data}}} + \underbrace{\lambda_r \frac{1}{N_r} \sum_{j=1}^{N_r} \left| \frac{du^\theta(x, t_j)}{dt} \right|_{t_j} + 2u^\theta(x, t_j)}_{\mathcal{L}_{\text{physics}}}^2 + \lambda_0 \underbrace{|u^\theta(x, 0) - 1|^2}_{\mathcal{L}_{\text{ic}}}$$



# The Complete PINN Loss

We want  $u^\theta(x, t)$  to simultaneously:

- 1 **Fit the data** at measurement points  $\{t_i, \hat{x}_i\}$
- 2 **Satisfy the ODE** at collocation points  $\{t_r\}$
- 3 **Satisfy the initial condition**  $u^\theta(x, 0) = x_0$

$$\mathcal{L}_{\text{total}}(\theta) = \underbrace{\lambda_d \frac{1}{N_d} \sum_{i=1}^{N_d} |u^\theta(x, t_i) - \hat{x}_i|^2}_{\mathcal{L}_{\text{data}}} + \underbrace{\lambda_r \frac{1}{N_r} \sum_{j=1}^{N_r} \left| \frac{du^\theta(x, t_j)}{dt} \right|_{t_j} + 2u^\theta(x, t_j)}_{\mathcal{L}_{\text{physics}}}^2 + \lambda_0 \underbrace{|u^\theta(x, 0) - 1|^2}_{\mathcal{L}_{\text{ic}}}$$

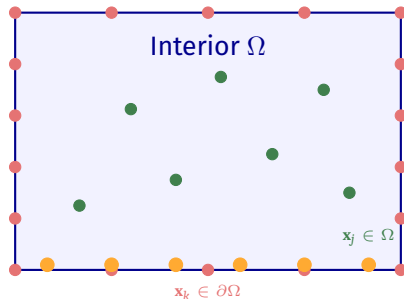
The physics loss needs **derivatives of the network output** — these are computed *exactly* via **automatic differentiation**.

# The Complete PINN Loss: General Case

$$\mathcal{L}_{\text{total}}(\theta) = \underbrace{\frac{\lambda_d}{N_d} \sum_{i=1}^{N_d} \left| u^\theta(\mathbf{x}_i, t_i) - \hat{u}_i \right|^2}_{\mathcal{L}_{\text{data}}} + \underbrace{\frac{\lambda_r}{N_r} \sum_{j=1}^{N_r} \left| \Psi[u^\theta](\mathbf{x}_j, t_j) - f(\mathbf{x}_j, t_j) \right|^2}_{\mathcal{L}_{\text{physics}}} \\ + \underbrace{\frac{\lambda_b}{N_b} \sum_{k=1}^{N_b} \left| \mathcal{B}[u^\theta](\mathbf{x}_k, t_k) - g(\mathbf{x}_k, t_k) \right|^2}_{\mathcal{L}_{\text{BC}}} + \underbrace{\frac{\lambda_0}{N_0} \sum_{m=1}^{N_0} \left| u^\theta(\mathbf{x}_m, 0) - u_0(\mathbf{x}_m) \right|^2}_{\mathcal{L}_{\text{IC}}}$$

| Symbol                                       | Meaning                                         | Evaluated on                    |
|----------------------------------------------|-------------------------------------------------|---------------------------------|
| $\Psi[\cdot]$                                | Differential operator (PDE/ODE)                 | Interior $\Omega \times (0, T]$ |
| $\mathcal{B}[\cdot]$                         | Boundary operator (Dirichlet / Neumann / Robin) | $\partial\Omega \times (0, T]$  |
| $f(\mathbf{x}, t)$                           | Forcing / source term                           |                                 |
| $g(\mathbf{x}, t)$                           | Prescribed boundary value                       |                                 |
| $u_0(\mathbf{x})$                            | Initial condition                               | $\Omega \times \{t = 0\}$       |
| $\lambda_d, \lambda_r, \lambda_b, \lambda_0$ | Loss weights (hyperparameters)                  |                                 |

# Sampling Sets for Each Loss Term



- Collocation (physics)
- Boundary (BC)
- Initial time  $t = 0$  (IC)
- Measurement (data, if available)

None of these sets require a mesh — points can be placed **anywhere**, including adaptively where the residual is largest.

# Two Modes: Forward and Inverse

## Forward Problem

**Given:** The PDE + BCs/ICs

**Find:** The solution  $u(x, t)$

Physics  $\longrightarrow$  Solution

## Inverse Problem

**Given:** Sparse measurements of  $u$

**Find:** Unknown PDE parameters  $\gamma$

Measurements  $\longrightarrow$  Physics

# Two Modes: Forward and Inverse

## Forward Problem

**Given:** The PDE + BCs/ICs

**Find:** The solution  $u(x, t)$

Physics  $\longrightarrow$  Solution

## Inverse Problem

**Given:** Sparse measurements of  $u$

**Find:** Unknown PDE parameters  $\gamma$

Measurements  $\longrightarrow$  Physics

**PINNs handle both in the same framework**  
(just add unknown parameters  $\lambda$  as trainable variables)

# Goal

We want data-efficient universal function approximators that naturally encode any underlying physical laws as prior information.

0 1 0 0 1 1 1 1 1 1 0 0 1 0 1 1 1 1 0 0  
0 1 1 1 1 1 1 0 1 1 1 0 0 1 1 1 0 1 0 1  
1 1 0 0 0 1 0 0 1 1 1 0 0 0 0 0 0 0 0 0

# Differentiable Physics

We use domain knowledge in the form of model equations.

- ⚡ We incorporate discretized version of this model equations into our training process.

In a supervised settings, we are interested in

$$f : X \rightarrow Y$$

Let's assume  $\Phi$  be a generic differential equations such that

$$\Phi : Y \rightarrow Z$$

which encodes a property of the solutions, e.g. some real world behavior we'd like to match.

In contrast to methods discussed in previous chapters, we use discretized version of  $\Phi$ , and employ it to guide the training of  $f$ .

## Example 1: Finding The Inverse Function of Parabola

Given  $\Phi : x \rightarrow x^2$ .

We need to find a function  $f \approx \Phi^{-1}$  such that  $\Phi(f(x)) = x$  (that is  $f(x) \approx \pm\sqrt{x}$ ). A neural network  $f_\theta$  is trained to approximate this.



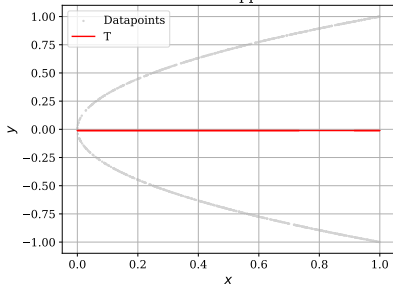
# Example 1: Finding The Inverse Function of Parabola

Given  $\Phi : x \rightarrow x^2$ .

We need to find a function  $f \approx \Phi^{-1}$  such that  $\Phi(f(x)) = x$  (that is  $f(x) \approx \pm\sqrt{x}$ ). A neural network  $f_\theta$  is trained to approximate this.

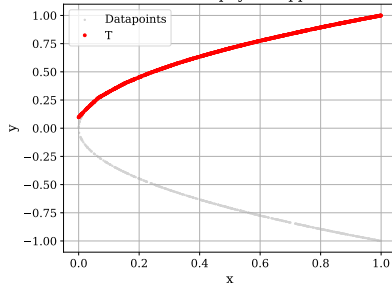
Data is  $\mathcal{D}\{(x^{(i)}, y^{(i)})\}_{i=1}^N$  where  $y^{(i)} = \pm\sqrt{x^{(i)}}$ .

Standard approach



$$\text{Loss: } \frac{1}{n} \sum_{i=1}^N (f_\theta(x^{(i)}) - y^{(i)})^2$$

Differentiable physics approach



$$\text{Loss: } \frac{1}{n} \sum_{i=1}^N (\Phi(f_\theta(x^{(i)})) - x^{(i)})^2.$$

Here, no  $y^{(i)}$  is used, but only the input  $x^{(i)}$  and known forward operator  $\Phi$ .

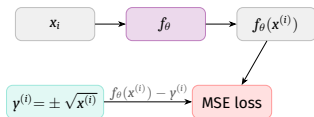
# Backpropagation in Physics Loss

For the physics loss, backprop through  $\Phi$  gives

$$\frac{\partial \mathcal{L}_{phys}}{\partial f_{\theta}(x^{(i)})} = 4f_{\theta}(x^{(i)})(f_{\theta}(x^{(i)})^2 - x^{(i)})$$

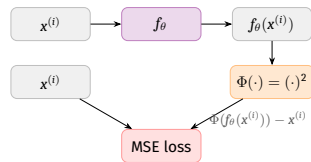
Its derivative vanishes at  $f_{\theta}(x^{(i)}) = 0$ , regardless of  $x^{(i)}$ . Hence, the zero function is a saddle point of the physics loss that doesn't affect the supervised loss.

**Supervised**



$$\mathcal{L}_{sup} = \frac{1}{n} \sum_{i=1}^N (f_{\theta}(x^{(i)}) - y^{(i)})^2$$

**Physics-based**



$$\mathcal{L}_{phys} = \frac{1}{n} \sum_{i=1}^N (\Phi(f_{\theta}(x^{(i)})) - x^{(i)})^2$$

no  $y^{(i)}$  labels used

Clearly, it didn't handle the **bifurcation**.

Can we do better?

Clearly, it didn't handle the **bifurcation**.

Can we do better?

We can learn the full distribution of the posterior.

Clearly, it didn't handle the **bifurcation**.

Can we do better?

We can learn the full distribution of the posterior.

Do you remember **Flow Matching**?

# Flow Matching for Inverse Function of Parabola

Assuming, normalized time, i.e.  $t \sim \mathcal{U}[0, 1]$ .

Recall, data is  $\mathcal{D}\{\mathbf{x}^{(i)} \equiv (x^{(i)}, y^{(i)})\}_{i=1}^N$  where  $y^{(i)} = \pm\sqrt{x^{(i)}}$ .

We define  $\mathbf{x} \sim \mathcal{N}(0, \mathbf{I}_2)$ . Initial condition  $\mathbf{x}_0^{(i)}[0] = x^{(i)}$  (we fix x-coordinate noise).

For every sample index  $i$ , the end point should be  $\mathbf{x}_1^{(i)} = (x^{(i)}, y^{(i)})^\top$ .

As we need to define a probability path  $p_t$ , but as we understand the  $p_t$  being marginal is difficult to compute.

Conditional interpolant can help in this case. We fix a specific pair and define a deterministic path between them, such as,

$$\mathbf{x}_t|_{(\mathbf{x}_0, \mathbf{x}_1)} = (1 - t)\mathbf{x}_0 + t\mathbf{x}_1$$

**But remember that the interpolant doesn't have to be linear.**

# Flow Matching for Inverse Function of Parabola

The marginal interpolant  $p_t(\mathbf{x})$  is recovered by averaging over all pairs:

$$p_t(\mathbf{x}) = \int p_t(\mathbf{x}_t | (\mathbf{x}_0, \mathbf{x}_1)) q(\mathbf{x}_0) p_{data}(\mathbf{x}_1) d\mathbf{x}_0 d\mathbf{x}_1$$

And, we have target conditional vector field as  $u_t = \frac{dx_t}{dt}$ , which is always deterministic, that is  $x_1 - x_0$  (only applicable for linear interpolant, for non-linear interpolant, you will see  $t$  in RHS of  $u_t$ ).



We can define a neural network  $v_\theta(\mathbf{x}_t, t)$  that will regress  $u_t$  with Flow Matching Loss function:

$$\mathcal{L}_{FM}(\theta) = \frac{1}{n} \sum_{i=1}^N \|v_\theta(\mathbf{x}_t^{(i)}, t^{(i)}) - u_t^{(i)}\|^2$$

# Flow Matching for Inverse Function of Parabola

At the inference time (prediction with the trained model), given  $\mathbf{x}_0 = (x^{(i)}, z)^\top$  with  $z \sim \mathcal{N}(0, 1)$ , the integrate the learned ODE model with step size  $\Delta t = \frac{1}{T}$ :

$$\mathbf{x}_{t_{k+1}}^{(i)} = \mathbf{x}_{t_k}^{(i)} + \Delta t v_\theta(\mathbf{x}_{t_k}^{(i)}, t_k)$$

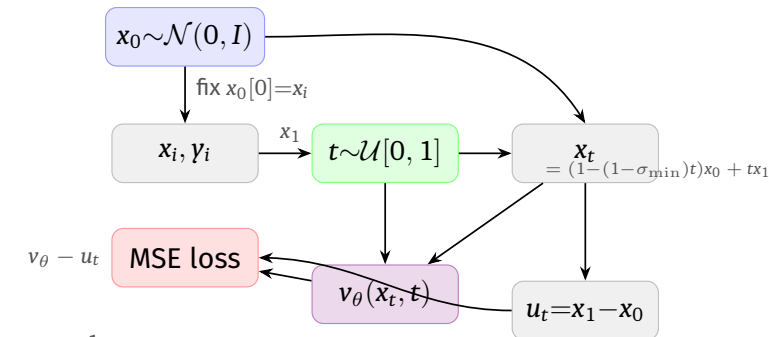
and then reimpose the x-conditioning at every step:  $\mathbf{x}_{t_{k+1}}^{(i)}[0] \leftarrow x^{(i)}$  and we get the second coordinate of the final state

$$\mathbf{x}_{t_1}^{(i)}[1] \sim p(y|x^{(i)})$$



# Flow Matching for Inverse Function of Parabola

## Flow Matching



$$\mathcal{L}_{\text{FM}} = \frac{1}{n} \sum_i \|v_\theta(x_t^{(i)}, t^{(i)}) - u_t^{(i)}\|^2$$

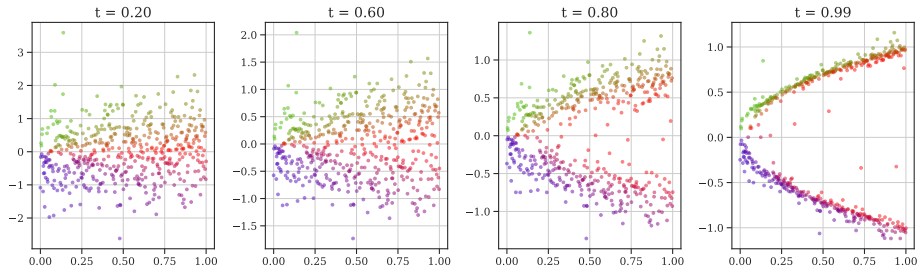
*no  $y_i$  labels;  $t, x_0$  stochastic*

# Flow Matching for Inverse Function of Parabola

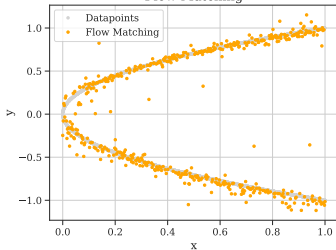
Caveats about Flow Matching:

- ⚡ Flow Matching is **not** PINN.
- ⚡ Physics get introduced in data construction  $y^{(i)} = \pm\sqrt{x^{(i)}}$  but FM never sees  $\Phi$ .
- ⚡ It learns conditional distribution  $p(y|x)$ .
- ⚡ In the case of FM, the parabola  $y^2 = x$  is the **support manifold** of that distribution, but no operator passed to the loss.

# Flow Matching for Inverse Function of Parabola



Flow Matching



Clearly, Flow Matching provides an approximate solution, although allows to take into account Bifurcation but makes mistakes as well.

**FM is not PINN.**



# Solving Nonlinear Partial Differential Equations

---

|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 0 | 1 | 0 | 0 | 1 | 1 | 0 | 1 | 0 | 0 | 1 | 0 | 0 | 0 | 1 | 0 | 1 | 1 |
| 1 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 0 | 1 | 1 | 1 | 1 | 0 | 0 | 1 |
| 1 | 0 | 0 | 1 | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 0 | 0 | 0 | 0 | 0 |

# Nonlinear Partial Differential Equation

Parameterized, nonlinear partial differential equation of the general form:

$$\Phi(\mathbf{x}, t, u, \partial_t u, \nabla u, \nabla^2 u, \dots) = 0, \quad (\mathbf{x}, t) \in \Omega \times (0, T]$$

where  $\Omega \subset \mathbb{R}^d$  be a spatial domain,  $t \in [0, T]$  be time, and  $u : \Omega \times [0, T] \rightarrow \mathbb{R}^m$  be the unknown field.  
We can have boundary condition

$$\mathcal{B}[u](\mathbf{x}, t) = 0, \quad \mathbf{x} \in \partial\Omega$$

and  
the initial condition as

$$u(\mathbf{x}, 0) = u_0(\mathbf{x}), \mathbf{x} \in \Omega$$

# Neural Parameterization

We parameterize the vector with a NN as  $u_\theta : \mathbb{R}^d \times [0, T] \rightarrow \mathbb{R}^m$  such as that  $u(\mathbf{x}, t) \approx u_\theta(\mathbf{x}, t)$ ,  $\theta \in \mathbb{R}^P$  for some  $P$ .

But this neural network need to be sufficiently smooth. So we cannot use ReLU. Instead we need to use tanh, sin, or Gaussian Error Linear Unit (GELU) activation functions.

$$GELU(x) = x \frac{1}{2} [1 + \text{erf}(x/\sqrt{2})]$$

Essentially, you can then write residual or error as

$$r_\theta(\mathbf{x}, t) := \Phi(\mathbf{x}, t, u_\theta, \partial_t u_\theta, \nabla u_\theta, \nabla^2 u_\theta, \dots)$$

# PINN Loss

We need to select points for regular region, initial condition, and boundary region. These points are called **collocation points**.

- 1  $\mathcal{S}_r \subset \Omega \times (0, T]$
- 2  $\mathcal{S}_b \subset \partial\Omega \times [0, T]$  \*(Here,  $\partial$  is not about derivative but about the boundary domain.
- 3  $\mathcal{S}_0 \subset \Omega \times \{0\}$

The PINN loss can then be defined as

$$\begin{aligned}\mathcal{L}(\theta) = & \frac{1}{|\mathcal{S}_r|} \sum_{(\mathbf{x}, t) \in \mathcal{S}_r} \|\Phi(\mathbf{x}, t, u_\theta, \partial_t u_\theta, \nabla u_\theta, \nabla^2 u_\theta, \dots)\|^2 \\ & + \frac{\lambda_b}{|\mathcal{S}_b|} \sum_{(\mathbf{x}, t) \in \mathcal{S}_b} \|\mathcal{B}[u_\theta](\mathbf{x}, t)\|^2 \\ & + \frac{\lambda_0}{|\mathcal{S}_0|} \sum_{(\mathbf{x}, t) \in \mathcal{S}_0} \|u_\theta(\mathbf{x}, 0) - u_0(\mathbf{x})\|^2\end{aligned}$$



## Example: Burgers' Equation

$$\Phi(u, \partial_t u, \partial_x u, \partial_{xx} u) := \partial_t u + u \partial_x u - \nu \partial_{xx} u = 0, \quad (x, t) \in [-1, 1] \times [0, T]$$

$\nu > 0$  is called Kinematic viscosity. The nonlinearity is by  $u \partial_x u$ .

### Boundary condition:

$\mathcal{B}[u](x, t) := u(x, t), \quad x \in \{-1, 1\}, t \in [0, T]$ . One choice is  $u(-1, t) = 0$ , and  $u(1, t) = 0$ .

### Initial condition:

$u(x, 0) = u_0(x) = -\sin(\pi x), \quad x \in [-1, 1]$ .

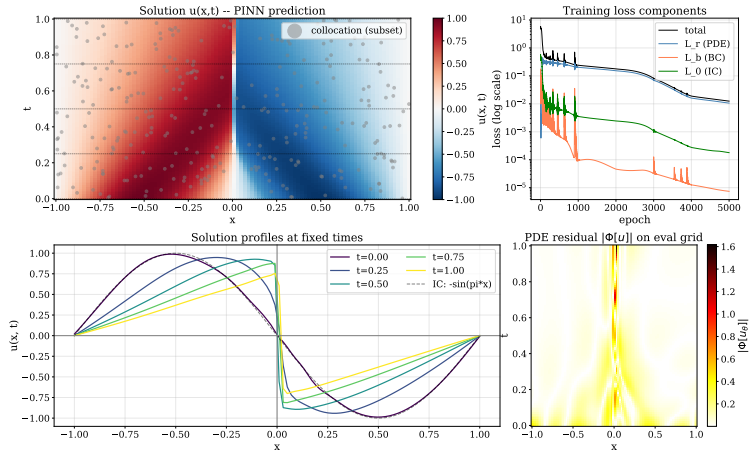
We also have collocation sets as  $\mathcal{S}_r \subset [-1, 1] \times (0, T]$ ,  $\mathcal{S}_b = \{-1, 1\} \times [0, T]$ , and  $\mathcal{S}_0 \subset [-1, 1] \times \{0\}$ .

# Derivative Computation Via AutoDiff

```
def Phi(u_theta, x, t):  
    u      = u_theta(x, t)                # (B, 1)  
  
    u_t    = torch.autograd.grad(  
        u, t,  
        grad_outputs=torch.ones_like(u),  
        create_graph=True)[0]             # del_t u  
  
    u_x    = torch.autograd.grad(  
        u, x,  
        grad_outputs=torch.ones_like(u),  
        create_graph=True)[0]             # del_x u  
  
    u_xx   = torch.autograd.grad(  
        u_x, x,  
        grad_outputs=torch.ones_like(u_x),  
        create_graph=True)[0]             # del_xx u  
  
    return u_t + u * u_x - nu * u_xx      # Phi[u]
```

# Burgers' Equation Result

Burgers PINN  $\nu=0.0032$   $N_r=10000$   $N_b=200$   $N_0=200$  epochs=5000



# References

- ⚡ Physics-informed machine learning <https://doi.org/10.1038/s42254-021-00314-5>
- ⚡ Physics Informed Deep Learning (Part I): Data-driven Solutions of Nonlinear Partial Differential Equations <https://arxiv.org/abs/1711.10561>
- ⚡ Physics Informed Deep Learning (Part II): Data-driven Discovery of Nonlinear Partial Differential Equations <https://arxiv.org/abs/1711.10566>
- ⚡ Thuerey, Nils, Philipp Holl, Maximilian Mueller, Patrick Schnell, Felix Trost, and Kiwon Um. "Physics-based deep learning." arXiv preprint arXiv:2109.05237 (2021). <https://arxiv.org/abs/2109.05237>
- ⚡ Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations Author links open overlay panel <https://doi.org/10.1016/j.jcp.2018.10.045>
- ⚡ <https://github.com/maziarraissi/PINNs>
- ⚡ <https://github.com/prateekbhustali/Physics-Informed-Neural-Networks>
- ⚡ [https://github.com/nguyenkhoa0209/pinns\\_tutorial](https://github.com/nguyenkhoa0209/pinns_tutorial)
- ⚡ <https://annien094.github.io/PINNs-tutorial-MICCAI-2024/>
- ⚡ Physics-informed neural networks for PDE problems: a comprehensive review <https://doi.org/10.1023/a:1012784129883>
- ⚡ A hands-on introduction to Physics-Informed Neural Networks for solving partial differential equations with benchmark tests taken from astrophysics and plasma physics <https://arxiv.org/pdf/2403.00599>